

Comparador de Farmacias Chile

El mismo remedio, hasta 20× más barato

Documentación técnica del proyecto

Plataforma web que compara el precio de un mismo medicamento entre las principales cadenas farmacéuticas de Chile (Cruz Verde, Salcobrand y Ahumada), agrupando los resultados por principio activo y mostrando el ahorro exacto.

Autor: Tu Nombre

github.com/Faraysito/farmacias-cl

Demo: farmacias-cl.vercel.app

19 de junio de 2026

Índice general

1. Resumen ejecutivo **5**

2. Contexto y problema **6**
2.1. El problema 6
2.2. Impacto 6
2.3. Objetivos 6

3. Solución propuesta **7**
3.1. Qué hace la plataforma 7
3.2. Flujo de uso 7

4. Arquitectura del sistema **9**
4.1. Componentes 9

5. Stack tecnológico **11**

6. Modelo de datos **12**
6.1. Descripción de las tablas 12
6.2. Definición de los modelos 13

7. Backend **15**
7.1. Estructura del código 15
7.2. Endpoints de la API 15
7.3. Estrategia de scraping por cadena 15
7.4. Normalización de productos 16
7.5. Persistencia y deduplicación diaria 16
7.6. Búsqueda bajo demanda y cálculo de ahorro 17
7.7. Programador nocturno 17

8. Frontend **18**
8.1. Tecnología y estructura 18
8.2. Sistema de diseño 18
8.3. Elemento de firma: la escalera de precios 19
8.4. Funcionalidades, accesibilidad y SEO 19

9. Seguridad **20**

10. Despliegue e infraestructura **21**
10.1. Componentes desplegados 21
10.2. Por qué Cruz Verde se desactiva en producción 21

11. Calidad: pruebas e integración continua **22**

12. Decisiones técnicas **23**

13. Deuda técnica y trabajo futuro **24**
13.1. Hecho 24
13.2. Pendiente 24
13.3. Deuda técnica conocida 24

14. Conclusiones **25**

A. Variables de entorno **26**

B. Comandos de puesta en marcha **27**

Índice de tablas

1.1. Ficha rápida del proyecto. 5

5.1. Tecnologías del backend y su justificación. 11
5.2. Tecnologías del frontend y la infraestructura. 11

7.1. Endpoints expuestos por el backend. 16
7.2. Técnica de scraping por cadena. 16

Índice de figuras

3.1. Lógica de scraping bajo demanda en el endpoint de búsqueda. 8

4.1. Arquitectura general del sistema. 9

6.1. Modelo entidad–relación. 12

Capítulo 1

Resumen ejecutivo

El **Comparador de Farmacias Chile** es una aplicación web que resuelve un problema concreto del mercado farmacéutico chileno: el mismo medicamento puede costar enormemente distinto según la cadena y la marca, y no existe una forma sencilla de compararlo. La plataforma permite buscar un remedio una sola vez y ver, lado a lado, el precio en las tres grandes cadenas, agrupado por principio activo, incluyendo alternativas bioequivalentes más baratas y el ahorro exacto en pesos y porcentaje.

El sistema se compone de un **backend** en Python con FastAPI que obtiene los precios mediante *scraping* (una técnica distinta por cadena según sus defensas), los normaliza y los almacena en PostgreSQL; y un **frontend** en Astro que entrega una interfaz rápida, accesible y con modo oscuro. Todo el stack corre en infraestructura gratuita (Vercel, Render y Neon) y se mantiene actualizado mediante un trabajo programado en GitHub Actions.

Tabla 1.1: Ficha rápida del proyecto.

Dominio	Salud / comparación de precios de medicamentos (Chile)
Backend	Python 3.12, FastAPI, SQLAlchemy 2, Alembic
Base de datos	PostgreSQL 16 + PostGIS (Neon) con `pg_trgm`
Scraping	Playwright, Algolia API, httpx + BeautifulSoup
Frontend	Astro, Tailwind CSS v4, Alpine.js, Chart.js
Infraestructura	Vercel, Render, Neon, GitHub Actions, UptimeRobot
Cadenas	Cruz Verde, Salcobrand, Ahumada

Capítulo 2

Contexto y problema

2.1. El problema

En Chile, un mismo principio activo puede tener una variación de precio de hasta 20 veces dependiendo de la cadena y de la marca comercial. Un caso representativo es el Losartán 50 mg, que puede ir desde aproximadamente $1.495 hasta $32.207 por el mismo principio activo y dosis. La mayoría de las personas no compara precios porque hacerlo es engorroso: cada cadena tiene su propio sitio web, sus propios nombres comerciales y sus propios laboratorios, y no hay un punto único que ponga la información en la misma mesa.

2.2. Impacto

Esta opacidad afecta especialmente a quienes tienen tratamientos crónicos (hipertensión, diabetes, colesterol), que compran los mismos medicamentos mes a mes. Una diferencia de varios miles de pesos por compra, repetida en el tiempo, representa un gasto evitable considerable. El bioequivalente correcto suele ser mucho más barato que la marca, pero el comprador rara vez sabe cuál es ni dónde está más económico.

2.3. Objetivos

Objetivo general

Construir una plataforma que permita comparar, de forma simple y gratuita, el precio de un medicamento entre las principales cadenas chilenas.

Objetivos específicos

- Obtener precios actualizados de Cruz Verde, Salcobrand y Ahumada.
- Normalizar y agrupar los productos por principio activo y dosis, de modo que marcas distintas equivalentes aparezcan juntas.
- Destacar el ahorro (absoluto y porcentual) y resaltar los bioequivalentes.
- Ofrecer histórico de precios y alertas cuando un producto baje de un umbral.
- Lograrlo sobre infraestructura gratuita y mantenible.

Capítulo 3

Solución propuesta

3.1. Qué hace la plataforma

La aplicación recibe el nombre de un medicamento o un principio activo y devuelve *grupos* de productos equivalentes. Cada grupo reúne todas las presentaciones con el mismo principio activo y dosis, ordenadas de menor a mayor precio, indicando la cadena, el laboratorio, si es bioequivalente y un enlace directo a la compra. En la cabecera de cada grupo se muestra el ahorro: la diferencia entre la oferta más barata y la más cara, en pesos y en porcentaje.

3.2. Flujo de uso

1. El usuario escribe un término (con autocompletado de principios activos y nombres comerciales).
2. El backend revisa si hay datos recientes en la base de datos.
3. Si no los hay o son insuficientes, dispara *scraping bajo demanda* de las cadenas activas en paralelo.
4. Los resultados se normalizan, se agrupan y se calcula el ahorro.
5. El frontend muestra cada grupo como una "escalera de precios".
6. El usuario puede filtrar, ordenar por ahorro, ver el histórico, crear una alerta de precio o ir a comprar.

La Figura 3.1 resume la lógica de scraping bajo demanda.

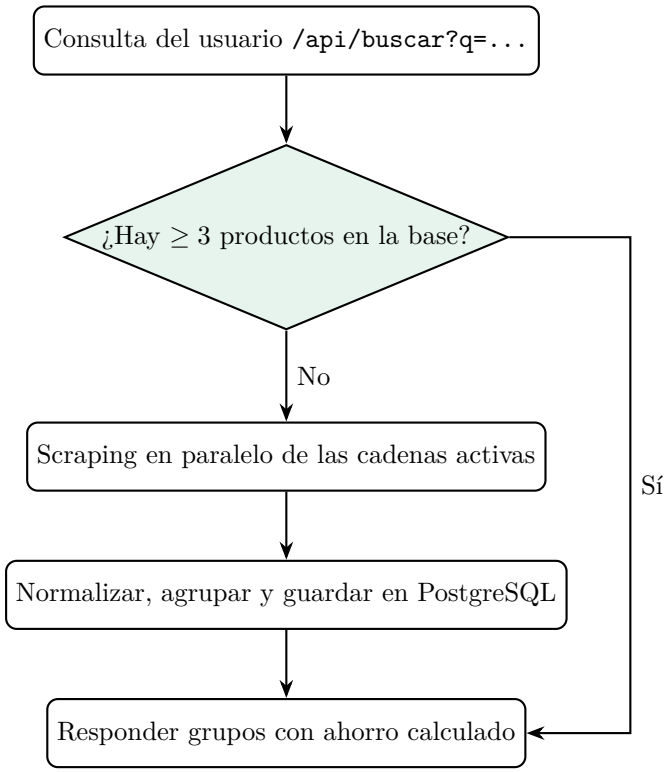


Figura 3.1: Lógica de scraping bajo demanda en el endpoint de búsqueda.

Capítulo 4

Arquitectura del sistema

El sistema sigue una arquitectura cliente–servidor clásica con una capa de adquisición de datos (scrapers) y un proceso programado de actualización. La Figura 4.1 muestra los componentes y su relación.

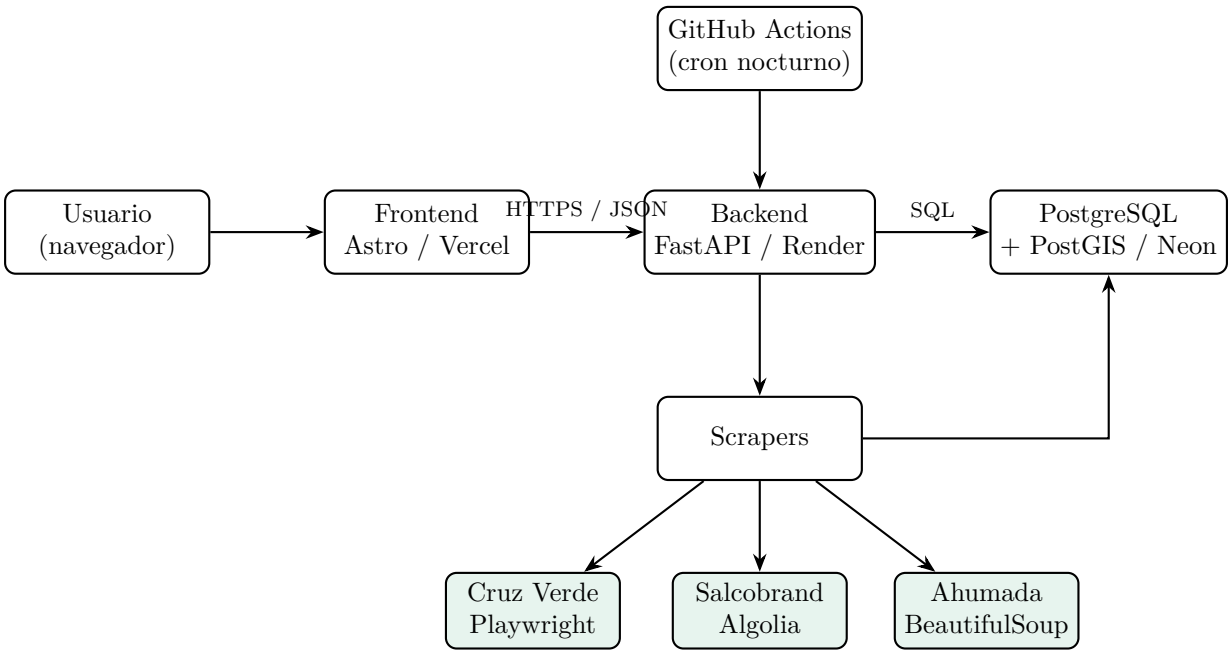


Figura 4.1: Arquitectura general del sistema.

4.1. Componentes

Frontend (Astro / Vercel).
Sitio estático que consume la API por HTTPS. Renderiza el buscador, los grupos de resultados, los modales de histórico y alertas, y las páginas de detalle y "acerca de".

Backend (FastAPI / Render).
Expone la API REST, ejecuta el scraping bajo demanda, normaliza y agrupa los datos, calcula el ahorro y gestiona alertas.

Base de datos (PostgreSQL + PostGIS / Neon).
Persiste principios activos, medicamentos, farmacias, precios (con histórico) y alertas. PostGIS queda preparado para una fase futura de farmacias cercanas por geolocalización.

Scrapers.
Módulos independientes, uno por cadena, con una interfaz común.

Programador (GitHub Actions).
Un *cron* nocturno despierta el backend y dispara la actualización del catálogo más consultado.

Capítulo 5

Stack tecnológico

Tabla 5.1: Tecnologías del backend y su justificación.

Tecnología	**Rol y motivo**
FastAPI	Framework asíncrono; ideal para orquestar varios scrapers en paralelo.
SQLAlchemy 2 + Alembic	ORM y migraciones versionadas de la base de datos.
PostgreSQL + PostGIS	Motor robusto con soporte geoespacial para el futuro.
psycopg 3	Driver moderno de PostgreSQL.
Playwright	Navegador real para Cruz Verde (protegida por Imperva).
httpx + BeautifulSoup	Cliente HTTP asíncrono y parser HTML para Ahumada.
RapidFuzz	Coincidencia aproximada de nombres de laboratorio.
slowapi	Limitación de tasa (rate limiting) por IP.
Resend	Envío de correos de alerta (opcional).
`pg_trgm`	Índices trigram para acelerar las búsquedas por texto.

Tabla 5.2: Tecnologías del frontend y la infraestructura.

Tecnología	**Rol y motivo**
Astro	Generador de sitios estáticos, muy rápido y con buen SEO.
Tailwind CSS v4	Estilos con configuración CSS-first mediante `@theme`.
Alpine.js	Interactividad ligera sin necesidad de un framework pesado.
Chart.js	Gráficos del histórico de precios.
Vercel	Hosting del frontend con *deploys* automáticos.
Render	Hosting del backend (capa gratuita).
Neon	PostgreSQL serverless gratuito.
GitHub Actions	Cron de scraping nocturno e integración continua (CI).
UptimeRobot	Ping periódico para evitar el arranque en frío de Render.

Capítulo 6

Modelo de datos

El esquema gira en torno a la entidad `Precio`, que registra cuánto cuesta un `Medicamento` en una `Farmacia` en una fecha dada. Cada medicamento referencia un `PrincipioActivo`, lo que permite agrupar marcas equivalentes. La Figura 6.1 muestra el modelo entidad–relación.

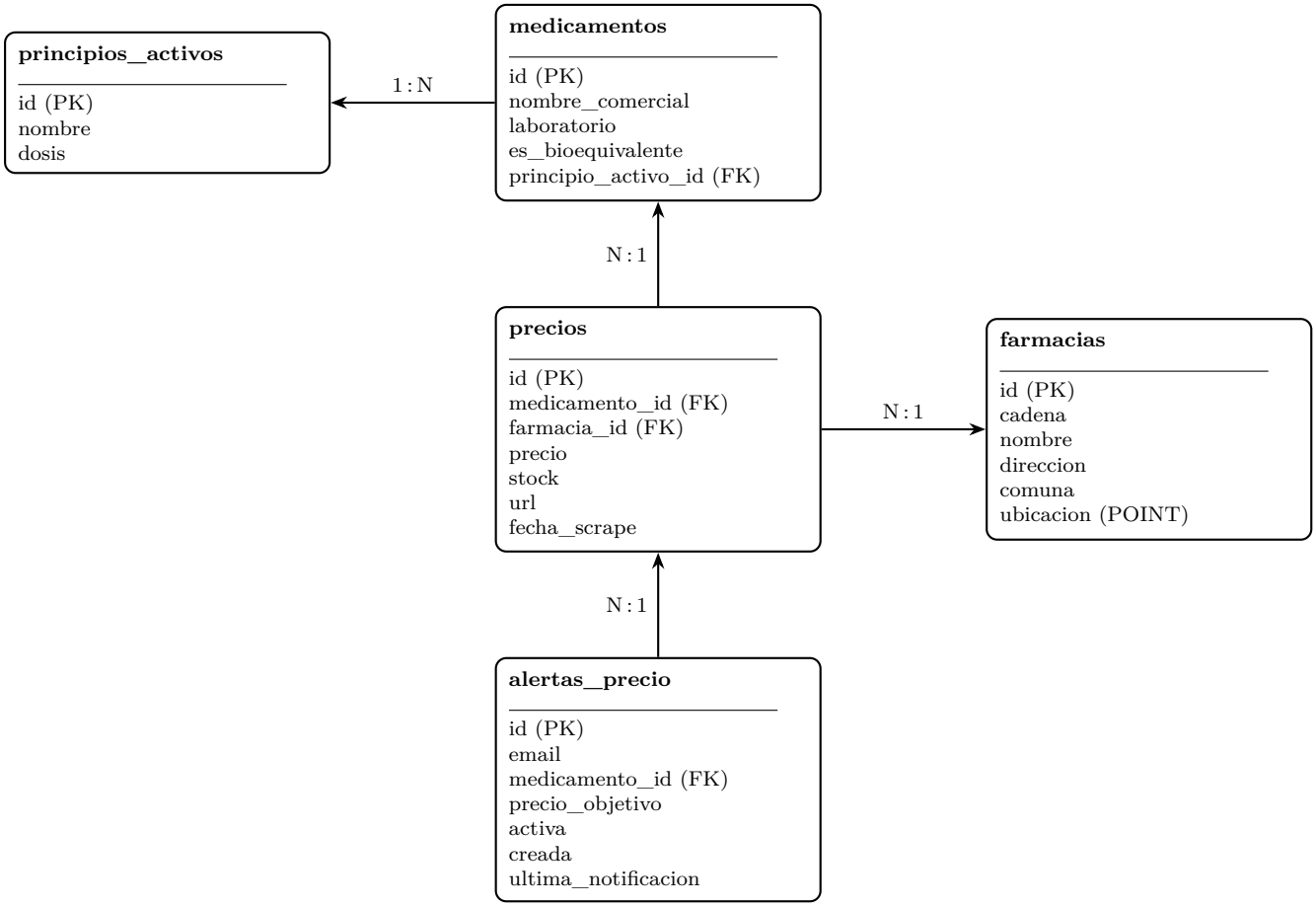


Figura 6.1: Modelo entidad–relación.

6.1. Descripción de las tablas

`principios_activos`.
Sustancia activa y su dosis. Es la clave de agrupación de productos equivalentes.

medicamentos.
Cada presentación comercial: nombre, laboratorio y si es bioequivalente. Apunta a su principio activo.

farmacias.
Punto de venta. En la fase actual se usa una farmacia "online" por cadena; el campo `ubicacion` (PostGIS) queda listo para farmacias físicas.

precios.
Lectura de precio de un medicamento en una farmacia con su fecha. Es la base del histórico.

alertas_precio.
Suscripción de un usuario para ser avisado cuando un medicamento baje de un precio objetivo.

6.2. Definición de los modelos

El Código 6.1 muestra las entidades principales en SQLAlchemy. Se utiliza un *timestamp* con zona horaria (`timezone.utc`) para evitar la función `datetime.utcnow()`, marcada como obsoleta en Python 3.12.

```python
from datetime import datetime, timezone
from sqlalchemy import (Boolean, Column, DateTime, Float,
                        ForeignKey, Integer, String)
from sqlalchemy.orm import relationship
from geoalchemy2 import Geography
from app.database import Base

def utcnow() -> datetime:
    # Timestamp con zona horaria (reemplaza datetime.utcnow)
    return datetime.now(timezone.utc)

class Medicamento(Base):
    __tablename__ = "medicamentos"
    id = Column(Integer, primary_key=True)
    nombre_comercial = Column(String, nullable=False, index=True)
    laboratorio = Column(String)
    es_bioequivalente = Column(Boolean, default=False)
    principio_activo_id = Column(Integer, ForeignKey("principios_activos.id"))
    principio_activo = relationship("PrincipioActivo", back_populates="medicamentos")
    precios = relationship("Precio", back_populates="medicamento")

class Precio(Base):
    __tablename__ = "precios"
    id = Column(Integer, primary_key=True)
    medicamento_id = Column(Integer, ForeignKey("medicamentos.id"), index=True)
    farmacia_id = Column(Integer, ForeignKey("farmacias.id"))
    precio = Column(Float, nullable=False)
    stock = Column(Boolean, default=True)
    url = Column(String)  # enlace directo al producto
    fecha_scrape = Column(DateTime, default=utcnow, index=True)

class AlertaPrecio(Base):
    __tablename__ = "alertas_precio"
    id = Column(Integer, primary_key=True)
    email = Column(String, nullable=False, index=True)
    medicamento_id = Column(Integer, ForeignKey("medicamentos.id"), index=True)
    precio_objetivo = Column(Float, nullable=False)
    activa = Column(Boolean, default=True, index=True)
    creada = Column(DateTime, default=utcnow)
    ultima_notificacion = Column(DateTime, nullable=True)
```

Código 6.1: Modelos SQLAlchemy (extracto de `app/models.py`).

Capítulo 7

Backend

7.1. Estructura del código

```
backend/
  main.py                 # App FastAPI: CORS, rate limit, cabeceras, routers
  app/
    config.py             # Configuracion leida del entorno
    database.py           # Engine y SessionLocal
    ratelimit.py          # Limiter compartido (slowapi)
    models.py             # Entidades SQLAlchemy
    schemas.py            # Esquemas Pydantic (entrada/salida)
    normalizador.py       # nombre -> (principio_activo, dosis)
    persistencia.py       # get_or_create y guardado con dedup diaria
    scrape_orchestrator.py# Orquesta los scrapers en paralelo
    api/
      busqueda.py         # GET /api/buscar (paginado + ahorro)
      autocomplete.py     # GET /api/autocomplete
      historico.py        # GET /api/historico/{id} (+ resumen)
      home.py             # GET /api/home/categorias|top|stats
      alertas.py          # POST/DELETE /api/alertas
      admin.py            # POST /api/admin/scrape-top (token)
    scrapers/
      base.py             # Modelo comun ProductoRaw
      cruzverde.py        # Playwright
      salcobrand.py       # Algolia
      ahumada.py          # httpx + BeautifulSoup
  alembic/                # Migraciones
  tests/                  # pytest
```

Código 7.1: Árbol del backend (resumido).

7.2. Endpoints de la API

7.3. Estrategia de scraping por cadena

Cada cadena exige una técnica distinta según cómo entrega y protege sus datos (Tabla 7.2).

El Código 7.2 ilustra el caso más simple: una consulta directa al índice Algolia de Salcobrand, sin navegador.

```
async def buscar(query: str, limite: int = 24):
    url = f"{ALGOLIA_HOST}/1/indexes/*/queries"
    params = {
```

Tabla 7.1: Endpoints expuestos por el backend.

Método	Ruta	Descripción
GET	`/api/buscar`	Búsqueda paginada con ahorro por grupo; scraping bajo demanda si faltan datos.
GET	`/api/autocomplete`	Sugerencias por prefijo.
GET	`/api/historico/{id}`	Histórico por cadena y resumen (mín/máx/promedio/-tendencia).
GET	`/api/home/...`	Datos de la portada: categorías, top y estadísticas.
POST	`/api/alertas`	Crea una alerta de precio por correo.
DELETE	`/api/alertas/{id}`	Cancela una alerta (baja lógica).
POST	`/api/admin/scrape-top`	Scraping del catálogo top (cabecera `X-Admin-Token`).
GET	`/api/health`	Comprobación de estado.

Tabla 7.2: Técnica de scraping por cadena.

Cadena	Técnica	Motivo
Cruz Verde	Playwright (navegador)	API protegida por Imperva; requiere cookies generadas por JavaScript.
Salcobrand	httpx + API Algolia	Su buscador es un índice Algolia público de búsqueda.
Ahumada	httpx + BeautifulSoup	Renderizado en servidor (Salesforce Commerce Cloud).

```
        "x-algolia-api-key": SALCOBRAND_ALGOLIA_API_KEY,
        "x-algolia-application-id": SALCOBRAND_ALGOLIA_APP_ID,
    }
    body = {"requests": [{"indexName": ALGOLIA_INDEX,
                          "params": f"hitsPerPage={limite}&query={query}"}]}
    async with httpx.AsyncClient(timeout=15) as client:
        r = await client.post(url, params=params, json=body)
        r.raise_for_status()
        data = r.json()
    hits = data["results"][0].get("hits", [])
    return [_a_producto_raw(h) for h in hits]
```

Código 7.2: Scraper de Salcobrand vía Algolia (extracto).

7.4. Normalización de productos

Los scrapers devuelven nombres heterogéneos (con marca, dosis, formas farmacéuticas y etiquetas como "(R)" o "(B)"). El normalizador transforma cada nombre en una tupla `(principio_activo, dosis)` mediante expresiones regulares y listas de términos a descartar. Esto es lo que permite agrupar productos equivalentes. Originalmente se planeaba usar el dataset del ISP (Instituto de Salud Pública), pero al estar fuera de línea se construyó un normalizador propio que cubre la mayor parte de los casos.

7.5. Persistencia y deduplicación diaria

Al guardar, se aplican funciones `get_or_create` para principios, medicamentos y farmacias, y se hace coincidir el laboratorio por su forma normalizada. Para construir un histórico limpio, solo se guarda **una lectura por día** por cada par (medicamento, farmacia): si ya existe una de hoy, se actualiza en lugar de insertar otra.

7.6. Búsqueda bajo demanda y cálculo de ahorro

El endpoint de búsqueda devuelve datos cacheados y, si hay menos de un umbral de productos, dispara el scraping en vivo. Por cada grupo se calcula el precio mínimo y máximo, y de ahí el ahorro absoluto y porcentual (Código 7.3).

```
@router.get("/buscar", response_model=BusquedaResponse)
@limiter.limit(RATE_LIMIT_BUSCAR)
async def buscar_medicamentos(request, q, pagina=1, por_pagina=20, db=Depends(get_db)):
    grupos = _buscar_en_db(db, q)
    total_productos = sum(g.cantidad_medicamentos for g in grupos)
    if total_productos < UMBRAL_PRODUCTOS_MIN:        # scraping bajo demanda
        await scrape_todas(q)
        grupos = _buscar_en_db(SessionLocal(), q)
    # paginacion sobre los grupos
    inicio = (pagina - 1) * por_pagina
    return BusquedaResponse(query=q, total=len(grupos), pagina=pagina,
                            por_pagina=por_pagina,
                            grupos=grupos[inicio:inicio + por_pagina])

# Calculo del ahorro por grupo:
#   ahorro_abs = precio_max - precio_min
#   ahorro_pct = (1 - precio_min / precio_max) * 100
```

Código 7.3: Núcleo del endpoint de búsqueda (extracto de `api/busqueda.py`).

7.7. Programador nocturno

El catálogo más consultado se mantiene fresco con un trabajo de GitHub Actions que se ejecuta de madrugada: hace un ping para despertar el backend en Render y luego llama al endpoint protegido `/api/admin/scrape-top` con un token. Así se evita pagar infraestructura adicional para tareas programadas.

Capítulo 8

Frontend

8.1. Tecnología y estructura

El frontend es un sitio Astro que se compila a estático y consume la API por HTTPS. La interactividad (búsqueda, autocompletado, filtros, modales, modo oscuro) se maneja con Alpine.js; los gráficos, con Chart.js.

```
frontend/src/
  layouts/Base.astro      # head, SEO/OpenGraph, fuentes, tema sin parpadeo, JSON-LD
  pages/
    index.astro           # Portada: buscador, resultados, modales
    medicamento.astro     # Detalle compartible (?id=)
    acerca.astro          # Contexto y aviso legal
  styles/global.css       # Tokens de diseno, modo oscuro, animaciones
```

Código 8.1: Árbol del frontend (resumido).

8.2. Sistema de diseño

El diseño usa un sistema de *tokens* semánticos donde el color codifica significado: el verde jade representa el precio bajo o el ahorro, y el tono arcilla, el precio alto. Se definen variables para modo claro y oscuro y se exponen como utilidades de Tailwind v4 (Código 8.2). La tipografía combina *Bricolage Grotesque* (titulares), *Plus Jakarta Sans* (texto) y *JetBrains Mono* (precios).

```
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));

@theme {
  --color-jade: #0e7c5a;        /* precio bajo / ahorro */
  --color-clay: #b9603a;        /* precio alto */
  --font-display: "Bricolage Grotesque", sans-serif;
  --font-mono: "JetBrains Mono", monospace;
}
.dark {
  --bg: #0c1410;
  --fg: #e8efea;
  --accent: #2fb67e;
}
```

Código 8.2: Tokens de diseño (extracto de `global.css`).

8.3. Elemento de firma: la escalera de precios

Cada grupo de resultados se presenta como una "escalera de precios": las ofertas se ordenan de menor a mayor, la más barata se resalta y cada fila incluye una barra que representa su precio relativo respecto del máximo. El Código 8.3 muestra la lógica que arma esas filas, aplica los filtros y asigna el color semántico.

```
filasDe(g) {
  const filas = [];
  for (const med of g.medicamentos) {
    if (this.filtros.soloBio && !med.es_bioequivalente) continue;
    for (const p of med.precios) {
      if (!this.filtros.cadenas.includes(p.cadena)) continue;
      filas.push({ med_id: med.id, nombre: med.nombre_comercial,
                   cadena: p.cadena, precio: p.precio, url: p.url });
    }
  }
  filas.sort((a, b) => a.precio - b.precio);
  const max = filas.length ? filas[filas.length - 1].precio : 1;
  filas.forEach((f, i) => {
    f.pct = Math.max(8, Math.round((f.precio / max) * 100));
    f.color = i === 0 ? "var(--accent-bright)"
            : (i === filas.length - 1 ? "var(--warn)" : "var(--fg-faint)");
  });
  return filas;
}
```

Código 8.3: Cálculo de las filas de la escalera de precios (Alpine.js).

8.4. Funcionalidades, accesibilidad y SEO

- Autocompletado con navegación por teclado y búsqueda con estado en la URL (parámetro `?q=`), lo que permite compartir y navegar con atrás/adelante.
- Filtros (solo bioequivalentes, por cadena) y orden por precio o por ahorro.
- Histórico de precios con gráfico y resumen; alertas de precio por correo.
- Página de detalle compartible por medicamento (`?id=`).
- Modo oscuro con conmutador persistente y sin parpadeo.
- Accesibilidad: etiquetas, foco visible, atributos `aria` en los modales y respeto por `prefers-reduced-motion`.
- SEO: meta-descripción, Open Graph, Twitter Card y datos estructurados (Schema.org).

Capítulo 9

Seguridad

- **Token de administración.** El endpoint de scraping se protege con un token comparado en tiempo constante (`secrets.compare_digest`) para no filtrar información por temporización.
- **Secretos fuera del código.** Las claves de Algolia y demás configuración se leen de variables de entorno.
- **Rate limiting.** Con slowapi se limita el endpoint de búsqueda y se aplica un límite global por IP.
- **Cabeceras de seguridad.** Se añaden `X-Content-Type-Options`, `X-Frame-Options` y `Referrer-Policy`.
- **CORS.** Orígenes configurables; `allow_credentials` desactivado porque no se usan cookies.

Capítulo 10

Despliegue e infraestructura

10.1. Componentes desplegados

Backend en Render.
Servicio web (capa gratuita). Arranque con `uvicorn main:app`. Variables: `DATABASE_URL`, `DISABLE_CRUZVERDE`, `ADMIN_TOKEN` y, opcionalmente, `RESEND_API_KEY`.

Frontend en Vercel.
Proyecto Astro con `PUBLIC_API_URL` apuntando al backend. Deploys automáticos en cada push.

Base de datos en Neon.
PostgreSQL serverless; la extensión PostGIS se habilita manualmente y `pg_trgm` la crea la migración.

GitHub Actions.
Dos flujos: el cron nocturno de scraping y la integración continua (lint, tests y build).

UptimeRobot.
Ping periódico a la API para mitigar el arranque en frío.

10.2. Por qué Cruz Verde se desactiva en producción

La capa gratuita de Render ofrece 512 MB de RAM, mientras que Playwright con Chromium necesita cerca de 1 GB. Por eso en producción se establece `DISABLE_CRUZVERDE=1` y solo quedan activas Salcobrand y Ahumada; Cruz Verde funciona en local. Una alternativa futura es ejecutar ese scraper desde un GitHub Action que escriba directamente en la base.

Capítulo 11

Calidad: pruebas e integración continua

El proyecto incluye pruebas con *pytest* (por ejemplo, casos parametrizados del normalizador y validación de los esquemas) y un flujo de integración continua que ejecuta el linter, las pruebas del backend y la compilación del frontend en cada push (Código 11.1).

```yaml
jobs:
  backend:
    steps:
      - run: pip install -r requirements-dev.txt
      - run: ruff check app tests
      - run: pytest -q
  frontend:
    steps:
      - run: npm ci
      - run: npm run build
```

Código 11.1: Flujo de CI (extracto de `.github/workflows/ci.yml`).

Capítulo 12

Decisiones técnicas

Scraping bajo demanda.
Evita mantener un catálogo gigante siempre fresco; la primera búsqueda es lenta y las siguientes son instantáneas.

Una técnica por cadena.
Se elige la más barata y robusta según las defensas de cada sitio (navegador solo donde es imprescindible).

Cron en GitHub Actions.
Tareas programadas gratis, sin servidores extra.

Deduplicación diaria.
Un punto de precio por día mantiene el histórico limpio y útil para gráficos y alertas.

Índices trigram.
Permiten que las búsquedas por subcadena usen índice en lugar de recorrer toda la tabla.

Capítulo 13

Deuda técnica y trabajo futuro

13.1. Hecho

- Scrapers de las tres cadenas, normalizador y agrupación por principio activo.
- Scraping bajo demanda, interfaz de comparación y despliegue gratuito.
- Portada, autocompletado, filtros e histórico con gráficos.
- Cálculo de ahorro, alertas de precio, página de detalle, modo oscuro y SEO.
- Pruebas e integración continua.

13.2. Pendiente

- Integrar el dataset del ISP cuando vuelva a estar disponible.
- Llevar Cruz Verde a producción mediante un GitHub Action dedicado.
- Farmacias independientes con PostGIS (mapa de farmacias cercanas).

13.3. Deuda técnica conocida

- El normalizador es una lista de reglas manual; se reemplazará por el ISP.
- La coincidencia de laboratorios es lineal por producto; con un catálogo grande conviene resolverla en SQL.
- Las alertas solo notifican al instante si el precio ya está bajo el objetivo al crearlas; falta un proceso que revise las alertas activas y avise cuando el precio baje después.
- El borrado de alertas no tiene autenticación ni enlace de cancelación; en producción conviene un token de baja.

Capítulo 14

Conclusiones

El proyecto demuestra cómo resolver un problema cotidiano y con impacto económico real mediante una arquitectura sencilla y de bajo costo. La combinación de scraping adaptado a cada cadena, un modelo de datos orientado a la comparación por principio activo y un frontend cuidado en diseño y accesibilidad entrega una herramienta útil y mantenible. Quedan abiertas líneas de mejora claras (integración del ISP, alertas proactivas y geolocalización de farmacias), pero la base es sólida y está en producción de forma gratuita y continua.

Apéndice A

Variables de entorno

```
DATABASE_URL=postgresql+psycopg://usuario:clave@host/neondb?sslmode=require
DISABLE_CRUZVERDE=1            # 1 en produccion (sin Playwright)
ADMIN_TOKEN=...                # protege /api/admin/scrape-top
RATE_LIMIT_BUSCAR=20/minute
RATE_LIMIT_DEFAULT=120/minute
SALCOBRAND_ALGOLIA_APP_ID=...
SALCOBRAND_ALGOLIA_API_KEY=...
RESEND_API_KEY=                # vacio: las alertas no envian correo
ALERTAS_FROM_EMAIL=alertas@tudominio.cl
```

Código A.1: Variables de entorno del backend (`.env`).

Apéndice B

Comandos de puesta en marcha

```
# 1. Base de datos
docker compose up -d

# 2. Backend
cd backend
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
alembic upgrade head
uvicorn main:app --reload

# 3. Frontend
cd ../frontend
npm install
npm run dev
```

Código B.1: Entorno local.